arXiv:2312.07003v2 [cs.AI] 16 Jul 2025

RACER: RATIONAL ARTIFICIAL INTELLIGENCE CAR-FOLLOWING-MODEL ENHANCED BY REALITY *

Tianyi Li
Department of Civil Engineering
Saint Louis University
tianyili.ai@gmail.com

Alexander Halatsis
Department of Aerospace Engineering
University of Minnesota
500 Pillsbury Dr. SE, Minneapolis, MN 55455, USA.
Halat005@umn.edu

Raphael Stern
Department of Civil, Environmental, and Geo-Engineering
University of Minnesota
500 Pillsbury Dr. SE, Minneapolis, MN 55455, USA.
rstern@umn.edu

ABSTRACT

This paper introduces RACER, the Rational Artificial Intelligence Car-following model Enhanced by Reality, a cutting-edge deep learning car-following model, that satisfies partial derivative constraints, designed to predict Adaptive Cruise Control (ACC) driving behavior while staying theoretically feasible. Unlike conventional models, RACER effectively integrates Rational Driving Constraints (RDCs), crucial tenets of actual driving, resulting in strikingly accurate and realistic predictions. Against established models like the Optimal Velocity Relative Velocity (OVRV), a car-following Neural Network (NN), and a car-following Physics-Informed Neural Network (PINN), RACER excels across key metrics, such as acceleration, velocity, and spacing. Notably, it displays a perfect adherence to the RDCs, registering zero violations, in stark contrast to other models. This study highlights the immense value of incorporating physical constraints within AI models, especially for augmenting safety measures in transportation. It also paves the way for future research to test these models against human driving data, with the potential to guide safer and more rational driving behavior. The versatility of the proposed model, including its potential to incorporate additional derivative constraints and broader architectural applications, enhances its appeal and broadens its impact within the scientific community.

Keywords Car-following · Automated vehicle (AV) · Driving behavior · Adaptive cruise control (ACC) · Deep learning · Artificial intelligence (AI)

1 Introduction

The revolutionary advances in vehicle automation have significant implications for the transportation sector. Numerous studies have delved into automated vehicle (AV) related topics such as enhanced traffic [1, 2, 3] and speed harmonization [4], among others. Although certain aspects of vehicle automation may augment traffic flow [5, 6], not all impacts are advantageous. For instance, existing research demonstrates that commercially available adaptive cruise control (ACC) vehicles may potentially decrease highway throughput [7]. These findings emphasize the significance of individual vehicles' driving characteristics in determining the aggregate behavior of the overall traffic flow. Consequently, accurately calibrated car-following models are an essential tool for analyzing the influence of ACC and AV dynamics on traffic flow and stability [8, 7, 9, 10, 11, 12].

* *Citation*: **Li, et al. RACER: Rational Artificial Intelligence Car-following-model Enhanced by Reality.**

While many different approaches have been proposed to model the vehicle-level dynamics of automated and partially automated vehicles [6, 9, 13], the majority of these models adapt existing car-following models to capture the dynamics of automated driving systems or AVs. While full automation in vehicles is likely to be a reality at some point in the future, current trends already showcase widespread usage of lower levels of autonomy such as driver-assist features like ACC. The introduction of new traffic dynamics from ACC vehicles is expected to impact traffic flow, even at low ACC market penetration rates [11, 7]. Furthermore, these dynamics would be contingent on the type of ACC vehicle and its driving patterns.

Since the early General Motors car following experiments in the 1950s [14], car-following (CF) behavioral modeling has been the focal point of the transportation community to decipher how human drivers react to their surrounding traffic environment. With the advent of deep learning, its application in car-following modeling has also seen significant traction [15, 16, 17, 18, 19, 20]. Deep learning [21], with its ability to learn hierarchical representations autonomously from raw data, has spurred a revolution in the way data is interpreted across diverse fields such as image recognition and natural language processing [22, 23, 24]. But, it also brings to the table certain challenges like overfitting and lack of interpretability, leading to the development of models like physics-informed neural networks (PINN) [25].

The CF modeling methods landscape is broadly divisible into three distinct groups [18]: physics-based models, data-driven models, and the emerging class of physics-guided AI car-following models. Physics-based models [11, 12, 6, 26, 27] rely on predefined mathematical functions with a limited number of parameters. These models simplify interactions between drivers and other vehicles, by using a simplified representation of human cognitive processes and mechanical dynamics. Data-driven models, on the other hand, have the unique capability to learn hierarchical representations directly from raw data, eliminating the need for explicit feature extraction or model formulation [15, 17, 16].

Recent years have witnessed the emergence of a combination of physics and data-driven models, offering a promising pathway to develop robust and interpretable CF models. These models strive to amalgamate domain-specific knowledge with deep learning prowess, thereby harnessing the strengths of both to create models adept at handling complex driving scenarios while adhering to real-world physics principles [25, 18, 19, 28, 29]. However, these models' significant limitation is their inability to comprehend a rational driver's actions in full. Both learning-based models and PINN works have overlooked Rational Driving Constraints (RDCs) [30], a set of conditions on car following behavior that are generally accepted as necessary for sensible or rational driving behavior, and traditional physics-based models such as OVRV can only represent a simplified version of reality, excluding many complexities in driving behavior [15, 18, 19, 17].

Our approach addresses these limitations by incorporating domain-specific knowledge and physical derivative constraints into the neural network architecture. The objective is to endow the model with a comprehensive understanding of rational driving behavior while still leveraging the advantages of data-driven learning. Through this integration, our aspiration is to create a more robust and interpretable car-following model that surpasses the capabilities of existing approaches, laying the groundwork for safer and more reliable autonomous driving systems.

The key contributions of this study are threefold: 1) the introduction of a novel deep learning methodology that integrates the RDCs in car-following modeling, providing a more flexible and efficient car-following modeling paradigm while addressing existing models' limitations; 2) the demonstration of our model's superior performance when compared with existing physics-based, data-driven, and PINN models; 3) a comprehensive analysis showing that our proposed model satisfies the RDCs, a feat yet to be achieved by other machine learning models.

The remainder of this article unfolds as follows: In Section 2, we introduce relevant literature at the intersection of car-following modeling and deep learning. We then explore various types of car-following models and elaborate on our proposed methodology and model in Section 3. Section 4 presents the experimental data utilized in this study followed by the experiments in Section 5. Subsequently, Section 6 provides a comprehensive discussion of our numerical experiment results and their implications. Finally, we conclude the article in Section 7, where we discuss the study's limitations and propose potential avenues for future research. To provide a clear understanding of the research problem and methodology, we present a comprehensive list of notations used throughout this paper in Table 1.

2 Review of Relevant Literature

In this section, we first delve into the literature encompassing physically-based car-following models, examining their evolution and impact. Subsequently, we pivot our attention towards the significant body of work wherein deep learning has been employed to model driving behavior, and where it has been ingeniously amalgamated with traditional models, to address their individual limitations and optimize performance.

Table 1: Nomenclature and acronyms used throughout the paper.

Symbol	Description	Unit
Variables and Parameters		
$a_t,\ \hat{a}_t$	True / predicted longitudinal acceleration	$\mathrm{m\,s^{-2}}$
$a_{\mathrm{NN}},\ a_{\mathrm{Phy}}$	Acceleration predicted by NN / by physical model	$\mathrm{m\,s^{-2}}$
$f_\theta(\cdot)$	Car-following mapping parameterized by θ	–
k_1, k_2	OVRV gain parameters	–
N	Number of data points in a batch	–
s_t	Space gap between follower and leader	m
$\hat{s}(t)$	Measured inter-vehicle spacing	m
v_t	Speed of follower vehicle	$\mathrm{m\,s^{-1}}$
$\hat{v}_\ell(t), \hat{v}_f(t)$	Measured lead / following vehicle speed	$\mathrm{m\,s^{-1}}$
$X_{\mathrm{seq}}, X_{\mathrm{phy}}$	Sequence input data / physical state input data	–
$\lambda_{1,2,3}$	Weights on RDC penalties in loss function	–
τ, η	OVRV time constant / jam distance parameters	s, m
θ	CF model parameters	–
$\mathcal{O},\ \mathcal{A}$	Observation space / action space	–
Acronyms		
ACC	Adaptive Cruise Control	–
AV	Automated Vehicle	–
CF	Car-Following	–
LSTM	Long Short-Term Memory network	–
MSE	Mean Squared Error	–
OVRV	Optimal Velocity–Relative Velocity model	–
PINN	Physics-Informed Neural Network	–
RDC	Rational Driving Constraint	–
RMSE	Root Mean Squared Error	–

2.1 The Modeling of Driving Behavior

The quest to model individual vehicle dynamics has been an intriguing focus for researchers since the mid-20th century [14]. The pivotal premise energizing this research area is that the conduct of one vehicle (dubbed as the "following" vehicle in this context) is significantly influenced by the actions of the vehicle preceding it (the "lead" vehicle).

In general, the following vehicle's acceleration at a specific time, $\ddot{x}(t)$, is expressed as a second-order ordinary differential equation, hinging on a selection of parameters: inter-vehicle spacing $s(t)$, the following vehicle speed $v(t) = \dot{x}(t)$, and the relative speed or inter-vehicle velocity $\dot{s}(t) = \dot{x}_\ell(t) - v(t)$ [31]. In this expression, $\dot{x}_\ell(t) = v_\ell(t)$ signifies the lead vehicle's speed, whereas x represents the distance or position from a predetermined starting point.

These well-known car-following models (CFMs) have evolved significantly over time [32, 33]. Classic models, such as the Gazis-Herman-Rothery (GHR) model [34] and Optimal Velocity (OV) model [35], offer fairly simple mathematical formulations for the acceleration function f. In contrast, modern models like the Intelligent Driver Model (IDM) [26] and Full Velocity Difference Model (FVDM) [27] incorporate additional parameters to accommodate subtler driving behaviors. These models establish deterministic relationships that accurately capture the impact of a lead vehicle's actions on a following vehicle, integrating aspects such as inter-vehicle distance, the speed of the following vehicle, and the relative speed between both vehicles.

Nevertheless, these models often face criticism for their oversimplification and inability to adapt to unique driving behavior [36]. Given these models are founded on predetermined assumptions and parameters, they may inadequately capture the variability and intricacy of real-world driving behaviors, especially in dynamic traffic scenarios [37].

In response to these limitations, contemporary research is gravitating towards the integration of machine learning techniques for modeling driving behavior. These machine learning models, capable of learning directly from data, hold the potential to adapt to the patterns and variability inherent in individual drivers' behavior. This shift has spurred the creation of innovative models such as the PINN based car-following models.

2.2 The Realm of Deep Learning and Driving Behavior

In the realm of driving behavior modeling, deep learning algorithms have increasingly been utilized to extract intricate car-following behaviors directly from trajectory data [38]. Furthermore, deep reinforcement learning has proven effective, particularly for large-scale simulations of naturalistic driving environments involving multiple objects [39, 40]. Certain studies, such as the neural network-based controller for ACC vehicles, have demonstrated superior performance in minimizing the discrepancy between real trajectories and model predictions [41]. However, they bring along a unique set of challenges. Deep learning models typically require extensive data and are vulnerable to overfitting, particularly when used in control settings, leading to less versatile control models [41]. Furthermore, the output generated by these models often lacks interpretability, leading to their perception as "black boxes" [42]. In control environments, these "black boxes" play a crucial role in the car's unpredictable behavior, where the addition of more layers may lead to less safe behavior [43].

To circumvent these challenges, hybrid models such as PINN-CFM have been conceived [18, 19, 28]. By infusing physics-based components into the deep learning architecture, these models boost their versatility and interpretability. This integration cleverly combines the strengths of both deep learning and physics-based modeling, while simultaneously curbing some of their inherent limitations. Cherian's study revealed that increasing the neuron count augments driving comfort but at the cost of compromised safety and elevated speed deviation. This underscores the criticality of integrating physical constraints into the neural network training process. It ensures that safe and comfortable driving predictions are based on the car's actual behavior, rather than solely depending on the training data [43].

Despite this progress, contemporary models still harbor certain limitations. Firstly, they typically rely on a specific physical car-following model, such as the IDM, in their loss functions, striving to find the best fit for physical car-following models [18, 19, 28]. Secondly, they fall short of fully accounting for the variation among drivers. Moreover, these models may fail to adhere to RDCs [30].

In response to these limitations, this study introduces a novel deep-learning model rooted exclusively in the partial derivative constraints of the RDCs. This model presents a more flexible and efficient constraint compared to those used in PINN-CF models. Consequently, we introduce a rational neural network-based car-following model, potentially transcending the capabilities of existing models.

3 Modeling Car-following Behavior

In this section, we propose a rational neural network-based car-following model designed to meet the RDCs while being capable of adapting to varying driving conditions and unique driver behaviors. We will detail the model's architecture, data processing, training process, and alignment with the RDC. Before delving into the proposed model, we first present an overview of the car-following model and three distinct types of modeling techniques. The classification of the driving policy mapping models includes engineering, deep learning (DL)-based, and physics-guided AI car-following models.

A car-following model is a mathematical mapping parameterized by $\boldsymbol{\theta}$, represented as $\boldsymbol{f}_\theta(\cdot|\theta)$, that maps state variables (observations of the traffic environment) $o \in \mathcal{O}$ to actions $a \in \mathcal{A}$ (longitudinal accelerations):

$$f_\theta : o \to a \tag{1}$$

While a vast body of literature employs various variables as inputs, this study focuses on three, namely the spacing $s(t)$, relative speed $\Delta v(t)$, and the subject vehicle's velocity $v(t)$ at the current time step. The state variable $\boldsymbol{o}$ constitutes a 3-dimensional state vector: $o = (s(t), \Delta v(t), v(t))$. The acceleration $a(t)$ of a following vehicle at time t is depicted as a second-order ordinary differential equation, a function of the inter-vehicle spacing $s(t)$, the following vehicle speed $v(t) = \dot{x}(t)$, and the relative speed or inter-vehicle velocity $\dot{s}(t)$. Here, x symbolizes the position or distance from an arbitrary starting point, and the derivative relationships are:

$$v(t) = \dot{x}(t) \tag{2}$$

$$\Delta v(t) = \dot{s}(t) \tag{3}$$

$$a(t) = \dot{v}(t) = \ddot{x}(t) \tag{4}$$

This CFM can then be formulated as:

$$\ddot{x}(t) = f_\theta(s(t), \Delta v(t), v(t)) \tag{5}$$

3.1 Engineering Car-following Model

The input of the Physics-based Car-following Model (Phy-CFM) or Engineering Car-following Model includes all possible signals $s(t)$ at time t from neighboring vehicles in the traffic environment, and its output is the acceleration $a(t + \Delta t)$ at the subsequent time step. The Phy-CFM model aims to find an optimal set of parameters θ^* that best fits real-world data to minimize the error between a simulated trajectory for a following vehicle and the observed vehicle trajectory as measured in a dataset. One approach that has proven to be successful at finding best-fit model parameter values is to minimize the mean square error (MSE) in inter-vehicle spacing between the simulated car following trajectory and the inter-vehicle spacing in the experimental data [44, 20]. The procedure can be written as:

$$\begin{aligned} &\underset{\theta}{\text{minimize}} : \textstyle\sum_{t=0}^{N} (\hat{a}(t) - a_{Phy}(t))^2 \\ &\text{s.t.} \quad a(t + \Delta t) = f_\theta(s(t), \Delta v(t), \dot{v}(t)), \quad t = 0, \Delta t, \ldots, N \end{aligned} \tag{6}$$

In this context, $\hat{a}(t)$ denotes the actual acceleration at time t, $a_{Phy}(t)$ represents the acceleration predicted by the physical model at time t, $f_\theta(\cdot)$ is the physical model of the prediction function parameterized by θ, Δt is the time step, and N is the size of the provided observed data. In most scenarios, the optimization problem in Equation (6) can be efficiently solved via numerical optimization.

3.2 Deep Learning based Car-following Model

While physics-based mappings are typically represented by mathematical formulas, deep learning-based mappings are often embodied through deep neural networks. The Artificial Neural Network (ANN) is a commonly used function approximator for car-following behavior. It considers states as inputs and generates the corresponding accelerations as outputs. Within each layer, every node linearly combines the outputs from the preceding layer and applies a non-linear activation function σ, passing it to the subsequent layer.

The mathematical representation of an ANN is as follows:

$$a_{NN} = \sigma(I) = \sigma\left(\boldsymbol{W}_L\left(\ldots\left(\boldsymbol{W}_1\boldsymbol{o} + \boldsymbol{b}_1\right)\cdots + \boldsymbol{b}_{L-1}\right) + \boldsymbol{b}_L\right) \tag{7}$$

Where $\sigma(\cdot)$ denotes the activation function, $\boldsymbol{W}_i$ represents the weight matrix for the i-th layer, $\boldsymbol{b}_i$ is the bias for the i-th layer, and $\boldsymbol{o}$ refers to the input vector to the network. This equation illustrates the network's transformation, i.e., the output of the network given the input state $\boldsymbol{o}$.

The application of machine learning techniques in modeling, classifying, and simulating car-following behaviors has garnered significant attention in recent years [15, 45, 46, 20, 47]. These advanced systems have demonstrated superior performance over traditional physical models. However, they often grapple with challenges of interpretability and may exhibit limitations in handling scenarios that diverge from their training datasets.

3.3 Physics-guided AI Car-following Model

A major shortcoming of learning-based models is their inability to consistently yield physically plausible outputs. This issue could lead to unsafe and irrational driving behaviors resulting in vehicle collisions in microscopic traffic simulations [48]. However, the recent development of PINNs offers a promising path toward a data-driven modeling paradigm that leverages the strengths of both physical and deep learning models [25]. A step in this direction is the car-following model proposed by Mo et al., which combines physical models with learning-based techniques [18, 19]. The key distinction between PINN-CFM and purely neural network-based CFM lies in the loss function, modified to include a weighted sum of two components: the data discrepancy loss MSE_{NN} calculated by the neural network, and the physics discrepancy estimated using the physical car-following model MSE_{Phy}.

$$
\begin{aligned}
L_\theta &= \alpha \cdot MSE_{NN} + (1-\alpha) \cdot MSE_{Phy} \\
&= \alpha \cdot \left(\frac{1}{N} \sum_{i=1}^{N} \left\| \hat{a}(i) - a_{NN}^{(i)} \right\|^2 \right) \\
&+ (1-\alpha) \cdot \left(\frac{1}{N} \sum_{i=1}^{N} \left\| a_{NN}^{(i)} - a_{Phy}^{(i)} \right\|^2 \right)
\end{aligned} \tag{8}
$$

Here, L_θ denotes the loss function, MSE_{NN} and MSE_{Phy} represent the mean square errors for data discrepancy and physics discrepancy, respectively, α is the trade-off coefficient, and N represents the size of observed data. The actual acceleration and its estimate for the observed state i are denoted as $a(i)$. Finally, $a_{NN}^{(i)}$ and $a_{Phy}^{(i)}$ express the estimated accelerations from the neural network model and engineering models, respectively.

Nevertheless, the existing models fall short in accounting for RDCs. The final outputs of these models are solely determined by learning-based mechanisms, and thus, cannot guarantee the rationality of the learned model. As a result, scenarios with potential collisions and unsafe driving behaviors could still occur, signaling a critical area for further research and improvement [18, 28].

3.4 Deep Learning-Based Car-Following Model Integration of Rational Driving Constraints

Previous car-following models relying on machine learning, as referenced in [16, 49, 15], have exhibited high performance in certain metrics, notably the MSE in predicted accelerations. Despite their capabilities, these models, including those developed on PINN [18, 19, 28] and reinforcement learning [17], often neglect the integration of RDCs. This section aims to fill this gap by outlining a deep learning-based car-following model that efficiently integrates RDC. As far as we know, this is an innovative addition to the field

Rational Driving Constraints are critical safety constraints for car-following models that are used to control autonomous vehicles. They represent natural behavioral laws that all rational drivers would obey to maintain safe driving [30, 5]. Specifically, these constraints state that a rational driver would always try to:

- **Speed Constraint**: Ensure that the derivative of acceleration with respect to speed is non-positive, indicating that the vehicle does not accelerate at higher speeds. This is mathematically represented as:

$$
\frac{da}{dv} \leq 0 \tag{9}
$$

- **Spacing Constraint**: Ensure that the derivative of acceleration with respect to spacing is non-negative, indicating that the vehicle does not decelerate when the spacing increases. This is mathematically represented as:

$$
\frac{da}{ds} \geq 0 \tag{10}
$$

- **Relative Speed Constraint**: Ensure that the derivative of acceleration with respect to relative speed is non-negative, indicating that the vehicle does not decelerate when the relative speed increases. This is mathematically represented as:

$$
\frac{da}{dr} \geq 0 \tag{11}
$$

Implementing these constraints in learning-based car-following models is essential as they warrant that the model's predictions align with basic safe and rational driving principles. The proposed solution in the provided code introduces a novel approach to enforce RDCs by incorporating them into the loss function of the model. This effectively guides the model during training in learning predictions that satisfy RDCs.

Our proposed model termed the "RACER: Rational Artificial Intelligence Car-following-model Enhanced by Reality," is a synergistic combination of two integral components. The first is a neural network component that caters to the sequential patterns within the data. The second is the incorporation of RDCs within the loss function. This ensures

the model's predictions align with the real-world physics principles of vehicle operation. This is particularly critical for automated vehicles, which require adherence to rational and safe driving principles.

The RDCs are integrated into the model by formulating a custom loss function that ensures adherence to real-world physics principles. Specifically, the RDCs constraints are enforced by adding penalty terms into the loss function that measure the degree of violation of each constraint. Specifically, we define our loss as a combination of MSE and an RDCs-informed constraint term, defined as follows:

$$\begin{aligned} Loss_{RDCs} &= MSE_{NN} + \lambda \cdot MSE_{RDCs} \\ &= MSE(a_{pred}, a_{true}) + \lambda_1 \cdot RDC_{speed} \\ &+ \lambda_2 \cdot RDC_{spacing} + \lambda_3 \cdot RDC_{\text{relative speed}} \\ &= \frac{1}{N}\sum_{i=1}^{N}\left(a_{\text{true}}^{(i)} - a_{\text{pred}}^{(i)}\right)^2 \\ &+ \lambda_1 \cdot \left(\frac{1}{N}\sum_{i=1}^{N}\text{ReLU}\left(\frac{\partial a_{\text{pred}}^{(i)}}{\partial v^{(i)}}\right)\right) \\ &+ \lambda_2 \cdot \left(\frac{1}{N}\sum_{i=1}^{N}\text{ReLU}\left(-\frac{\partial a_{\text{pred}}^{(i)}}{\partial s^{(i)}}\right)\right) \\ &+ \lambda_3 \cdot \left(\frac{1}{N}\sum_{i=1}^{N}\text{ReLU}\left(-\frac{\partial a_{\text{pred}}^{(i)}}{\partial r^{(i)}}\right)\right) \end{aligned} \tag{12}$$

In this equation, $a_{\text{true}}^{(i)}$ and $a_{\text{pred}}^{(i)}$ represent the actual and predicted acceleration at the i^{th} time step, and λ is the trade-off coefficient respectively. The terms $\frac{\partial a_{\text{pred}}^{(i)}}{\partial v^{(i)}}$, $\frac{\partial a_{\text{pred}}^{(i)}}{\partial s^{(i)}}$, and $\frac{\partial a_{\text{pred}}^{(i)}}{\partial r^{(i)}}$ represent the derivatives of the predicted acceleration concerning velocity, spacing, and relative velocity at the i^{th} time step, respectively. Here, $v^{(i)}$, $s^{(i)}$, and $r^{(i)}$ correspond to velocity, spacing, and relative velocity at the i^{th} time step, respectively, while N signifies the total number of data points in a batch.

This loss function is crafted to minimize the MSE between the predicted and true accelerations while also respecting the RDCs, mediated by the hyperparameters λ_1, λ_2, and λ_3. The constraints are modeled as RDC_{speed}, $RDC_{spacing}$, and $RDC_{\text{relative speed}}$, which are computed as the derivatives of the model's output corresponding to the driving parameters. By applying the ReLU function, these quantities are ensured to be non-positive.

In our car-following model, let's denote the complex function that maps inputs like speed (v), spacing (s), and relative speed (r) to the predicted acceleration (a_{pred}) as $f(x)$. The function $f(x)$ includes elementary operations, denoted as g, that could represent various transformations and activations in a deep learning model. These operations are the fundamental building blocks that we leverage when performing automatic differentiation, which is a process that follows the chain rule for differentiation in an efficient manner. Specifically, for an arbitrary function $f(x)$ that is composed of these elementary operations g, automatic differentiation is expressed as follows:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx} \tag{13}$$

In this process, the chain rule is recursively applied, enabling us to compute the derivatives of a_{pred} with respect to v, s, and r that form the basis for the computation of RDCs. Once these derivatives are obtained, we use the Rectified Linear Unit (ReLU) function [50], $\text{ReLU}(x) = max(0, x)$ to retain only those portions of the gradients that violate the rational driving constraints, as the RDCs are presented in Equations (9)-(11), with the ReLU function serving to retain only those gradients violating the RDCs, i.e., the positive velocity gradients and the negative gradients of spacing and relative speed. This innovative application of automatic differentiation and the ReLU function in the computation of RDCs allows the integration of these constraints into the loss function, thereby guiding the model's predictions to align more closely with the principles of vehicular movement.

Using the calculated gradients, the ReLU function is applied as follows:

$$RDC_{speed} = \text{ReLU}\left(\frac{\partial a_{\text{pred}}}{\partial v}\right) = \max\left(0, \frac{\partial a_{\text{pred}}}{\partial v}\right) \tag{14}$$

$$RDC_{spacing} = \text{ReLU}\left(-\frac{\partial a_{\text{pred}}}{\partial s}\right) = \max\left(0, -\frac{\partial a_{\text{pred}}}{\partial s}\right) \tag{15}$$

$$RDC_{\text{relative speed}} = \text{ReLU}\left(-\frac{\partial a_{\text{pred}}}{\partial r}\right) = \max\left(0, -\frac{\partial a_{\text{pred}}}{\partial r}\right) \tag{16}$$

These operations enforce the RDCs, retaining only those portions of the gradients that violate the constraints while nullifying the rest. By integrating these RDCs into the loss function, the model's predictions are guided to align with the principles of vehicular movement, producing more realistic results. This novel integration of domain-specific knowledge is a significant step towards developing more robust and reliable car-following models.

In our proposed model, two distinct inputs, denoted as X_{seq} and X_{phy}, are utilized. The X_{seq} input includes time-dependent variables (speed, spacing, and relative speed) and is processed through several layers of a neural network architecture that can handle sequence data to manage temporal dependencies in driving behavior. This processing allows us to capture the intricate temporal dynamics that occur in vehicle-following scenarios. On the other hand, X_{phy}, containing speed (v), spacing (s), and relative speed (r) at a particular timestep, is used to enforce the RDCs through the calculation of gradients. To apply the RDCs in our loss function, we compute the derivatives of a_{pred} with respect to v, s, and r. These derivatives represent the gradients of the prediction contingent on each variable in X_{phy}.

The overall architecture of the RACER model is illustrated in Fig. 1. As shown, the model processes two distinct data inputs: sequential data (X_{seq}) and physical state data (X_{phy}). The process begins with **(1) input encoding** of this data, which then flows into **(2) the neural network processing** core. This pathway generates **(3) an acceleration prediction (a_{pred})**. Simultaneously, this prediction, along with current state variables, is used for **(4) derivative computation**, enabling the calculation of both the MSE loss and the RDC loss. These are combined into **(5) a composite loss**, which is then used for **(6) backpropagation** to update the model parameters. This end-to-end process, guided by both forward data flow (solid arrows) and gradient/derivative information (dashed arrows), ensures both predictive accuracy and rational driving constraint compliance.

Algorithm 1 Enforcing Rational Driving Constraints

1: **Input:** Sequence of vehicle states X_{seq}, Physical vehicle states X_{phy}
2: **Output:** Predicted acceleration a_{pred}, Loss $\mathcal{L}$
3: $Z_{\text{seq}} \leftarrow NN_{\text{seq}}(X_{\text{seq}})$ ▷ LSTM encoding
4: $Z'_{\text{seq}} \leftarrow \text{process}(Z_{\text{seq}})$ ▷ Further processing
5: $Z'_{\text{phy}} \leftarrow \text{process}(X_{\text{phy}})$ ▷ Physical state processing
6: $a_{\text{pred}} \leftarrow \text{combine}(Z'_{\text{seq}}, Z'_{\text{phy}})$ ▷ Combine representations
7: Calculate gradients $\frac{\partial a_{\text{pred}}}{\partial v}$, $\frac{\partial a_{\text{pred}}}{\partial s}$, $\frac{\partial a_{\text{pred}}}{\partial r}$
8: $RDC_{\text{speed}} \leftarrow \text{ReLU}\left(\frac{\partial a_{\text{pred}}}{\partial v}\right)$ ▷ Speed constraint
9: $RDC_{\text{spacing}} \leftarrow \text{ReLU}\left(-\frac{\partial a_{\text{pred}}}{\partial s}\right)$ ▷ Spacing constraint
10: $RDC_{\text{rel_speed}} \leftarrow \text{ReLU}\left(-\frac{\partial a_{\text{pred}}}{\partial r}\right)$ ▷ Relative speed constraint
11: $\mathcal{L} \leftarrow \text{MSE}(a_{\text{pred}}, a_{\text{true}}) + \lambda_1 \cdot RDC_{\text{speed}} + \lambda_2 \cdot RDC_{\text{spacing}} + \lambda_3 \cdot RDC_{\text{rel_speed}}$ ▷ Combined loss
12: **return** $a_{\text{pred}}, \mathcal{L}$

The details of the algorithm are presented in Algorithm 1 and Fig. 1. The algorithm operates through several key stages: NN_{seq} represents the sequence handling neural network (line 3), process$(\cdot)$ in lines 4-5 signifies additional processing steps that may vary depending on the exact network architecture used, and combine$(\cdot)$ in line 6 refers to the operation that combines the processed sequence and physical inputs. The core innovation lies in using two distinct inputs: X_{seq} (temporal sequence data) and X_{phy} (current kinematic state data typically used in physical models). Lines 7-10 implement the RDC enforcement by computing partial derivatives and applying ReLU functions to penalize constraint violations. The final loss function (line 11) combines the MSE loss for prediction accuracy with the RDC loss for constraint compliance. This integration of domain-specific knowledge and sequence data modeling represents a crucial step toward developing more reliable car-following models.

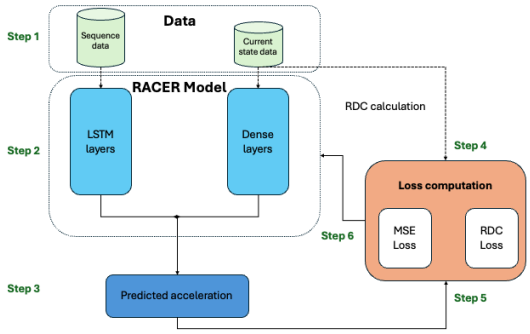
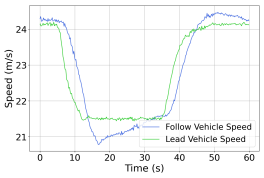
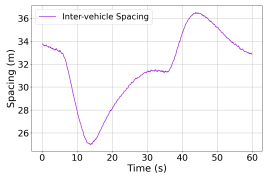


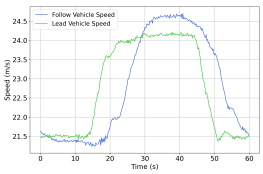
Figure 1: Architecture of the RACER model showing end-to-end dataflow: (1) input encoding of sequence data (X_{seq}) and kinematic states (X_{phy}); (2) neural network processing; (3) acceleration prediction (a_{pred}); (4) derivative computation for RDC calculation; (5) composite loss formation; (6) backpropagation. Solid arrows indicate forward flow; dashed arrows show gradient computations.



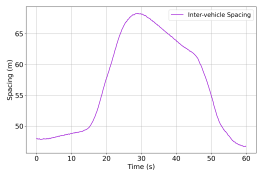
(a) Lead vehicle and follow vehicle speed (min setting).



(b) Inter-vehicle spacing (min setting).



(c) Lead vehicle and follow vehicle speed (max setting).



(d) Inter-vehicle spacing (max setting).

Figure 2: Comparison of vehicle trajectory data from the ACC car-following dataset under different ACC settings. (a) and (b) show the speed and spacing for the minimum gap ACC setting, while (c) and (d) show these for the maximum gap ACC setting.

4 Data Description

Our analysis employs a principal dataset derived from a sequence of car-following experiments conducted by Gunter et al. [11]. This dataset is amassed using a variety of commercially available vehicles equipped with ACC systems. Each of these ACC-activated vehicles adheres to a uniform testing procedure, where a leading vehicle traverses at a pre-established speed sequence for a set duration at each pace. The ACC vehicle, while trailing the lead vehicle, has its ACC active throughout the experiment. To accurately emulate car-following behavior, the acceleration of the trailing vehicle is determined by evaluating its speed change over a time interval, corresponding to some time steps as the data is captured at a rate of 10 Hz (0.1 s intervals). The whole dataset includes about 10 minutes of vehicle driving trajectory, resulting in more than 6000 rows of data.

To ensure accurate position and speed data, each vehicle is fitted with high-precision GPS receivers. This allows for the calculation of the inter-vehicle gap and relative speed. Fig. 2 provides a snapshot for one of the following vehicles under two conditions: minimum gap ACC setting, allowing the vehicle to maintain a close spacing to the one ahead, and maximum gap ACC setting, permitting greater spacing from the lead vehicle. This sample offers an illustrative example of leader-follower speed and spacing trajectories. The data collection operates at a 10 Hz sampling rate. Detailed information about the assorted set of vehicles utilized in Gunter et al.'s experiment [11] is provided in Table 2. We used 80% of the data for training, 10% for validation, and the remaining 10% for testing.

Table 2: Summary of ACC vehicles tested by [11].

Vehicle	Make	Style	Engine	Min. Speed (mph)
A	1	Full-size sedan	Combustion	25
B	1	Compact sedan	Combustion	25
C	1	Compact hatchback	Hybrid	25
D	1	Compact SUV	Combustion	25
E	2	Compact SUV	Combustion	0
F	2	Mid-size SUV	Combustion	0
G	2	Full-size SUV	Combustion	0

The ACC following vehicles are subjected to four distinct experimental conditions: oscillatory, low-speed steps, high-speed steps, and dips. The oscillatory trial is intended to assess the ACC system's behavior under fluctuating lead speed and headway conditions. The low-speed step trial aims to gather steady-state following behavior across an expansive range of low speeds. Similarly, the high-speed step trial collects steady-state driving trajectories at higher speeds. The dips trial evaluates the ACC vehicle's response to abrupt and substantial shifts in the lead vehicle's speed.

5 Numerical Experiments

This section presents the computational experiments undertaken to validate the efficacy of a rational neural network-based car-following model in car-following behavior modeling. We provide details of the experimental design, with a specific focus on the training, data assembly, and calibration of the physical parameters. The experiments are executed on a computing setup equipped with an Intel i7-12700K CPU @ 3.6 GHz processor complemented with 64 GB of memory. The deep learning-based models are trained and evaluated on a single NVIDIA GeForce RTX 3090 with 24 GB memory, utilizing the PyTorch 2.0 framework for these operations.

5.1 Physical Model

This study conducts experiments leveraging an experimental dataset collected by Gunter et al. during car-following trials with commercially available ACC vehicles, as discussed in Section 4. Specifically, our CF models are trained and validated using the oscillatory dataset. This dataset consists of car-following trajectories, where the lead vehicle executes pre-planned driving maneuvers while the trailing vehicle has its ACC activated. This precision of the GPS system used to collect the trajectory data allows for the measurement of the inter-vehicle space gap and relative speed with a fine granularity of 0.1 seconds.

To emulate car-following behavior precisely, the acceleration of the trailing vehicle is determined by evaluating its speed change over a time interval, corresponding to some time steps as the data is captured at a rate of 10 Hz (0.1 s intervals). The velocity of the trailing vehicle is represented as V_f. The equation for acceleration is:

$$\hat{a}(t) = \ddot{x}_f = \frac{V_f(t+0.1) - V_f(t)}{0.1} \tag{17}$$

Here, $\ddot{x}_f$ represents the acceleration of the trailing vehicle. We use a time interval as an example, the velocities $V_f(t+0.1)$ and $V_f(t)$ represent the trailing vehicle's velocity at time $t+0.1s$ and t respectively. The acceleration over the interval is the difference between these velocities divided by the time interval of 0.1 seconds.

In this study, we employ the OVRV, an extended model to the FVD Model [27]. We select the OVRV model to depict the ACC vehicle driving behavior because it has demonstrated its ability to effectively encapsulate car-following dynamics of commercially-available ACC vehicles with a straightforward model [51, 12, 44]. Additionally, we have reviewed other relevant literature [52, 53] and implemented widely used physical models such as IDM and Gipps for comparison with the OVRV model, using the same data and calibration methods. For the Gipps model, which relies on speed control instead of acceleration control, we estimated acceleration using headway, as outlined in the literature [52, 53], to ensure a fair comparison. The results show that the OVRV model performs best in both spacing and speed metrics, making it our chosen baseline physical model.

$$\ddot{x}_f = k_1(s - \eta - \tau v) + k_2 \dot{s} \tag{18}$$

The OVRV model is characterized by four model parameters that can be tuned to accurately represent car-following behavior. First, k_1 is the gain parameter on the constant effective time-gap term. Second, k_2 is a relative velocity parameter with respect to the lead vehicle. A higher k_2 value results in a quicker reaction to relative velocity changes. The τ parameter acts as a time constant, which is related to the inter-vehicle time gap. Finally, η represents the jam distance (i.e., inter-vehicle spacing at zero speed). For this study, we use $k_1 = 0.052$, $k_2 = 0.236$, $\tau = 0.796$, and $\eta = 13.836$ for minimum gap setting, and $k_1 = 0.018$, $k_2 = 0.105$, $\tau = 2.489$, and $\eta = 0.0003$ for maximum gap setting through calibration. The performance of the OVRV model, along with the simulation results, are presented in Section 6.

To calibrate the OVRV model, we minimize the MSE in inter-vehicle acceleration, using the equation:

$$\begin{aligned} \underset{\theta}{\text{minimize}}: \quad & \sqrt{\tfrac{1}{N}\textstyle\sum_0^N (\ddot{x}_{\mathrm{f}}(t) - \hat{a}(t))^2 dt} \\ \text{subject to:} \quad & s(t) = \hat{s}(t) \\ & v_{\mathrm{l}}(t) = \hat{v}_{\mathrm{l}}(t) \\ & v_{\mathrm{f}}(t) = \hat{v}_{\mathrm{f}}(t) \end{aligned} \tag{19}$$

Here, the $\hat{s}(t)$, $\hat{v}_{\mathrm{l}}$(t), and $\hat{v}_{\mathrm{f}}$(t) represent real data, and N denotes the number of data points in the training data. The optimization process in Equation 19 is solved via numerical optimization, ensuring that our model accurately predicts inter-vehicle acceleration based on the collected velocity data.

5.2 Deep Learning CF Model

In this follow-up experiment, we use the neural network modes in replicating the physical car-following behavior, specifically, we use the Long Short-Term Memory (LSTM) model, offering an alternative approach to modeling inter-vehicle dynamics. Incorporating the LSTM model into this approach is necessary because of its inherent capacity to retain long-term dependencies and accurately forecast future acceleration patterns of nonlinear data. LSTM has been proven to be capable of dealing with sequence data in several studies [54, 55], such as language process, and speech recognition, and shows that the LSTM model can model complex sequential interactions. The LSTM has also been applied in the field of transportation in a variety of studies [56, 57, 45, 18], including demand forecasting, car-following modeling, and trajectory reconstruction. Building on the success of our previous study that utilizes the OVRV model, we now delve into the realm of neural networks and deep learning techniques.

In our approach to crafting an accurate LSTM model, we consider acceleration calculations along with spacing, following velocity, and lead velocity as inputs. We establish an elaborate LSTM model architecture with five layers, each layer housing 64 LSTM units, complemented by an additional layer of dense connections. This meticulously designed architecture excels in capturing and learning the temporal dependencies that are intrinsic to the sequential car-following data, making it adept at modeling car-following dynamics.

The functioning of the LSTM layer is based on the following equations:

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_t] + b_f) \tag{20}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_t] + b_i) \tag{21}$$

$$\tilde{c}_t = \tanh(Wc \cdot [h_{t-1}, X_t] + b_c) \tag{22}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{23}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, X_t] + b_o) \tag{24}$$

$$h_t = o_t \odot \tanh(c_t) \tag{25}$$

In the above equations, σ is the sigmoid activation function, $\odot$ denotes element-wise multiplication, f_t, i_t, o_t are the forget, input, and output gates respectively, and $\tilde{c}_t$ is the candidate cell state. Wf, W_i, W_o, and W_c are the weight matrices and b_f, b_i, b_o, and b_c are the bias terms. These equations illustrate the information flow through an LSTM unit at a given time step t, for a specific input X_t, and previous hidden state h_{t-1} and cell state c_{t-1}. The output from the final LSTM layer and the last element of each input sequence are connected to a dense layer, leading to the formation of two parallel information streams. These streams are then merged and funneled through a final dense layer to generate the model output y_{pred}. To augment the model's robustness and accuracy, we deploy several essential strategies during the training phase. These include data shuffling to inhibit pattern memorization by the model and to bolster its generalization ability. We also adopt superior training practices such as early stopping and Z-score normalization to facilitate a more accurate and trustworthy representation of the original data.

5.3 PINN CF Model

Combining deep learning and the OVRV model, we create the PINN model, inspired by [18, 19]. Still, we use the OVRV as our physical model instead of the IDM model in their study since OVRV has been shown better for the ACC driving behavior modeling [12, 11]. Utilizing the same data as before, we edit the loss function of the deep learning model. From our previous experiment with the OVRV model, we use the most accurate calibration coefficients from the OVRV model from the previous model for k_1, k_2, τ, η respectively. We then use the predictions from the OVRV model with these coefficients to create a predicted acceleration that could be compared to the prediction deep learning output. By using these two predictions, we can create a loss function for the neural network by using the following equation that takes in the root mean squared error of the true and predicted data of the neural mse_{loss} and the physics error of the neural network prediction, which is the neural acceleration prediction minus the OVRV prediction $physics_{loss}$, with an alpha coefficient α weight that determines the balance between data discrepancy and physics discrepancy contributions:

$$Loss_{PINN} = \alpha \cdot MSE_{NN} + (1 - \alpha) \cdot MSE_{Physics} \tag{26}$$

During the model training process, we treat α as a hyper-parameter, enabling the neural network to actively modify its value and enhance the model's accuracy. Through iterative experimentation, we identify the α value that yields the optimal balance between minimizing data discrepancy and physics discrepancy. This approach leads to the development of a model with heightened accuracy.

5.4 RACER Model

The model proposed in this study leverages the same LSTM component as the deep learning and PINN models discussed earlier. However, as delineated in the methodology section, we utilize two distinct data inputs, X_{seq} and X_{phy}. The employment of X_{phy} data in conjunction with the modified model structure enables the computation of the predicted acceleration with respect to the input data comprising of velocity v, spacing s, and relative velocity r. Additionally, it allows us to enforce the RDCs during the model training process.

Our results demonstrate that the proposed model excels over the previously mentioned models in all performance metrics, including the predicted acceleration, and simulated spacing and speed, as gauged through simulation. This underscores the efficacy of our novel approach and paves the way for further improvements in modeling ACC driving behavior. The loss function deployed in this model is outlined in Equation (27), offering a balanced consideration of both the data and RDCs aspects of the model.

$$\begin{aligned} Loss_{RDCs} &= MSE_{NN} + \lambda \cdot MSE_{RDCs} \\ &= MSE(a_{pred}, a_{true}) + \lambda_1 \cdot RDC_{speed} \\ &+ \lambda_2 \cdot RDC_{spacing} + \lambda_3 \cdot RDC_{\text{relative speed}} \end{aligned} \tag{27}$$

6 Analysis of the Numerical Experiments

This section presents an analysis of numerical experiments conducted on four distinct models: the OVRV model, the NN model, the PINN model, and the RACER model. These models are evaluated using test data, which includes inter-vehicle spacing, following velocity, and leading velocity variables. The performance of these models is scrutinized

Table 3: Root mean squared errors for different models used as car-following controller with minimum gap setting.

	RACER	OVRV	NN	PINN
Acceleration (m/s^2)	**0.204**	0.208	0.209	0.207
Speed (m/s)	**0.09**	0.17	0.125	0.114
Spacing (m)	**0.261**	1.47	0.305	0.272

using data from ACC-equipped vehicles under both minimum and maximum gap settings, as previously described in Section 4.

6.1 Model Evaluation: Use the trained model as CF controller

As recommended by Punzo and Montanino [58], we prefer to compare model performance using the cumulative inter-vehicle spacing rather than the instantaneous values or speed error. Consequently, we take into account the cumulative error for the temporal evolution of states. Thus, to evaluate the model's performance, we use the trained or calibrated model as a car-following controller, and using our models' acceleration predictions, we reconstruct position and velocity trajectories. These trajectories (spacing and speed profiles) for the following vehicles are derived based on kinematic dynamics, using a time step of $\Delta t = 0.1$ s—aligned with the experimentally gathered data:

$$\begin{bmatrix} s \\ v \end{bmatrix}_{t+\Delta t} = \begin{bmatrix} s \\ v \end{bmatrix}_t + \begin{bmatrix} v_l - v \\ a_{pred} \end{bmatrix}_t \Delta t \tag{28}$$

In this study, we compare the generated trajectories with actual trajectories to assess discrepancies. To quantify errors in acceleration, inter-vehicle spacing, and velocity, we employ the root mean squared error (RMSE) metric. It's important to note that $a_{simulation}$, $s_{simulation}$, and $v_{simulation}$ represent the simulated acceleration, inter-vehicle spacing, and velocity, respectively. Conversely, a_{actual}, s_{actual}, and v_{actual} denote the corresponding actual measurements obtained from real-world data. The discrepancies are quantified as follows:

$$RMSE_{acceleration} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(a_{simulation,i} - a_{actual,i})^2}, \tag{29}$$

$$RMSE_{spacing} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(s_{simulation,i} - s_{actual,i})^2}, \tag{30}$$

$$RMSE_{velocity} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(v_{simulation,i} - v_{actual,i})^2}. \tag{31}$$

6.2 Model Evaluation: ACC Vehicle with Minimum gap Setting

Table 3 presents the RMSE for four evaluated models using data from the oscillatory experiment on Vehicle A under minimum and maximum gap settings. The RACER model demonstrates superior accuracy across all metrics, achieving the lowest RMSE in acceleration prediction, where all models perform comparably.

The performance of these models extends to predicting speed and spacing, which are crucial parameters that depend on the leading vehicle's behavior. The OVRV model exhibits the weakest performance; its error margins for speed are at least twice as high as those of the RACER model and deteriorate further for spacing. Conversely, the PINN and NN models predict vehicle spacing with greater precision, as shown in Fig. 5, and adapt more effectively to changes in the leading vehicle's speed, shown in Fig. 4. Nonetheless, the PINN and NN models' performance in speed and

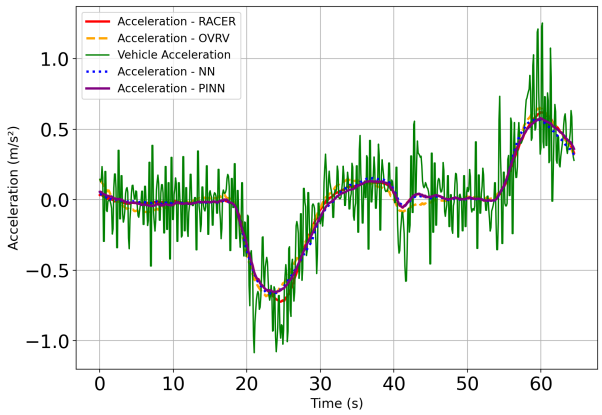


Figure 3: Comparison of vehicle acceleration predictions from different simulation models over time.

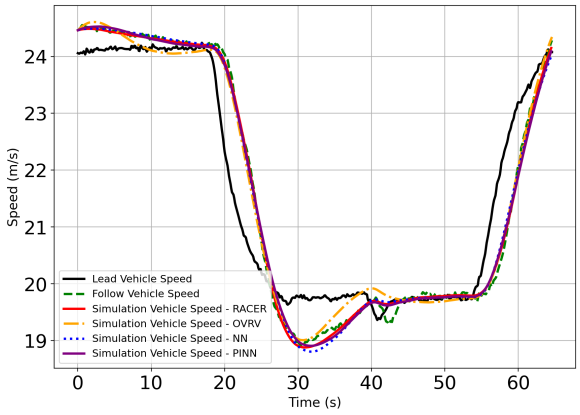


Figure 4: Comparison of vehicle velocity from different simulation models over time.

spacing prediction is not as consistent as the RACER model, with RMSE values exceeding those of the RACER model, indicating room for improvement.

We then scrutinize the predictions made by various deep learning models against testing data, specifically focusing on compliance with the RDCs. These constraints evaluate the model-predicted accelerations to three driving parameters: velocity (v), spacing (s), and relative speed (r). To accurately compute these constraints, we utilize PyTorch's automatic differentiation function, which allows us to compute the precise gradients of acceleration with respect to each variable. This approach ensures that the derivatives are computed exactly at each time step, rather than approximating them numerically. As such, any violations in the RDCs are detected based on the multivariate derivatives across velocity, spacing, and relative speed.

Fig. 6 presents the results for a baseline NN model. Intriguingly, the NN managed to fulfill the RDCs relative to speed and relative speed constraints but fell short in the context of spacing, exhibiting violations approximately 60% of the time, respectively. These violations are highlighted in the figure using red lines to indicate where the partial derivative $\frac{da}{dv} \leq 0$, signaling a violation of the RDC. Additionally, the model's performance with respect to velocity did not improve significantly, despite some minor non-violation segments.

A different perspective is provided in Fig. 7, which shows the performance of a PINN. Although the PINN model also complied fully with the RDCs of relative speed constraint, it exhibited a stark contrast when it came to velocity and spacing constraints, with an approximately 50% violation rate for both these parameters.

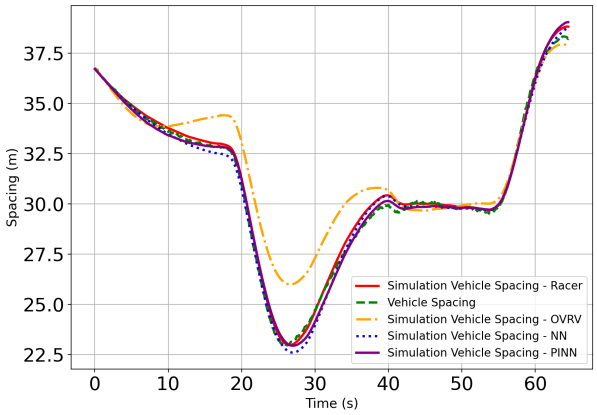
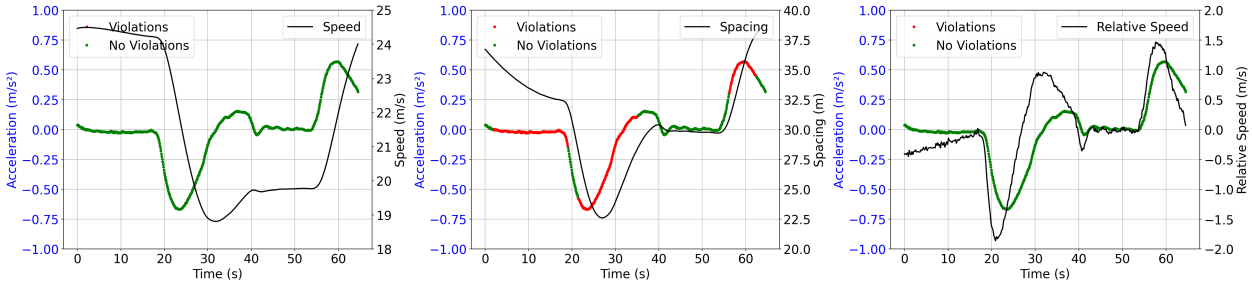
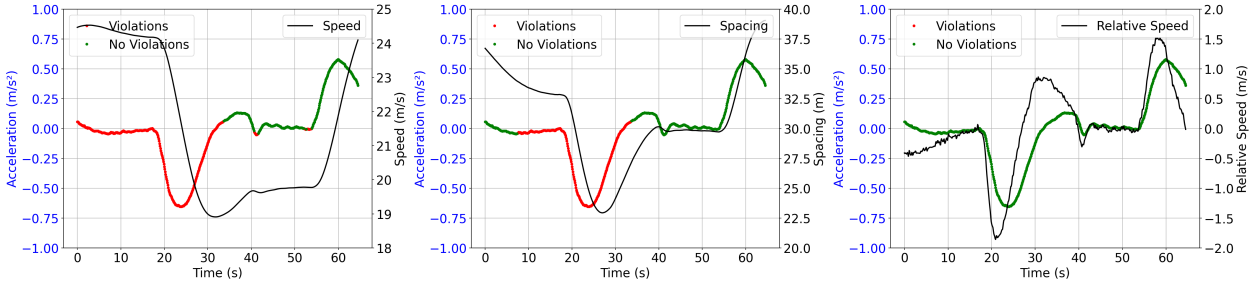


Figure 5: Comparison of vehicle spacing from different simulation models over time.



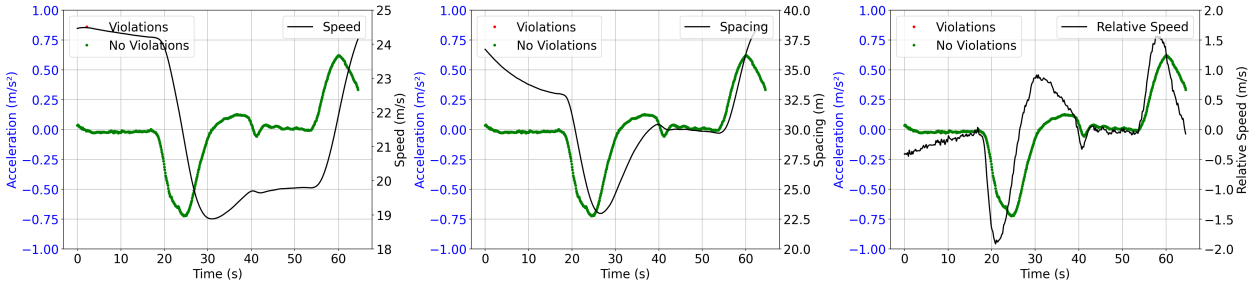
(a) RDC Violations - Speed **(b) RDC Violations - Spacing** **(c) RDC Violations - Relative Speed**

Figure 6: Illustrations of predictions and violations by the LSTM Neural Network model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.



(a) RDCs Violations - Speed **(b) RDCs Violations - Spacing** **(c) RDCs Violations - Relative Speed**

Figure 7: Illustrations of predictions and violations by the physics-informed Neural Network model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.



(a) RDC Violations - Speed **(b) RDC Violations - Spacing** **(c) RDC Violations - Relative Speed**

Figure 8: Illustrations of predictions and violations by the RACER model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.

Table 4: Root mean squared errors for different models used as car-following controllers with maximum gap setting.

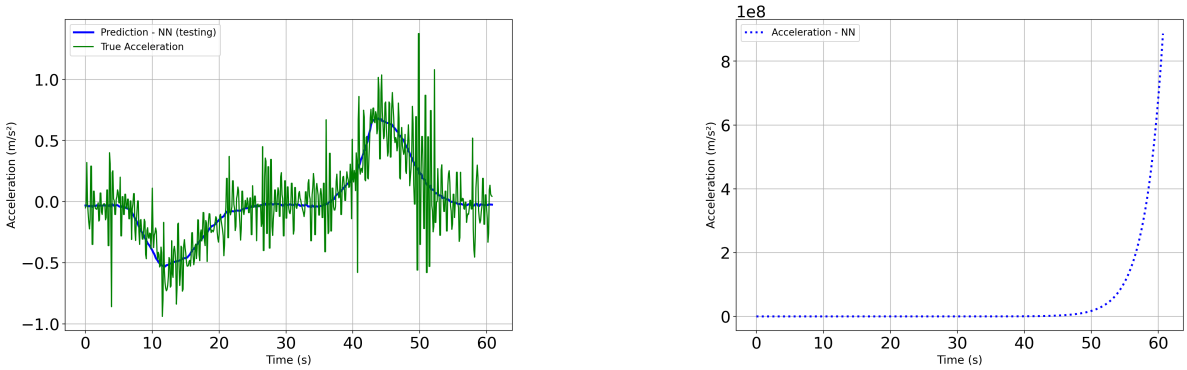
	RACER	OVRV	NN	PINN
Acceleration (m/s^2)	**0.221**	0.614	N/A (crash)	0.227
Speed (m/s)	**0.081**	1.839	N/A (crash)	0.268
Spacing (m)	**0.394**	22.761	N/A (crash)	2.573

Finally, we turn our attention to our proposed model, the Rational Neural Network, the results of which are delineated in Fig. 8. Using the same automatic differentiation approach for detecting RDC violations, this network demonstrates flawless performance, with zero RDC violations for relative speed, velocity, or spacing. Unlike the other models, this network successfully adheres to the constraints across all variables. This outcome highlights the efficacy of our model in adhering to real-world driving principles, further reinforcing the benefits of integrating domain-specific knowledge into deep learning models for more realistic and reliable predictions. The superior performance of RACER can be attributed to three key mechanisms: (i) the explicit enforcement of RDCs prevents the model from learning physically implausible behaviors that lead to prediction errors, (ii) the dual-input architecture (X_{seq} and X_{phy}) allows the model to leverage both temporal patterns and instantaneous physical constraints simultaneously, and (iii) the gradient-based RDC enforcement through automatic differentiation ensures that the learned mapping respects fundamental driving principles at every prediction step, leading to more stable and realistic long-term trajectory simulation.

The OVRV model's poor performance in spacing prediction (RMSE of 1.47 m vs. RACER's 0.261 m) stems from its simplified linear relationships that cannot capture the complex, nonlinear dynamics present in real ACC behavior. The standard Neural Network's high violation rates (60% for spacing constraints) occur because it optimizes purely for prediction accuracy without physical constraints, leading to mathematically optimal but physically implausible solutions that violate basic driving principles. The PINN model's moderate violations (50%) result from the soft enforcement of physics through the loss function—while it encourages physics compliance, it cannot guarantee it, allowing violations when data fitting objectives conflict with physical principles.

6.3 Model Evaluation: ACC Vehicle with Maximum Setting

The performance of the tested models under the maximum gap ACC setting diverges notably from that observed under the minimum gap setting. The RACER model maintains consistent efficacy, replicating its minimum gap setting proficiency as indicated in Table 4. The RACER model surpasses other models, especially in spacing and speed metrics. In particular, the performance of the NN model drastically declines in the maximum gap setting scenario, resulting in collisions during the simulation. In contrast, the NN model experiences a significant decline in performance under the maximum gap scenario, resulting in collisions during the simulation. While the NN model's predictions align well with the training data, as shown in Fig. 9a, its application as a car-following controller leads to erratic and unstable behavior, as depicted in Fig. 9b. This highlights a key insight: a model's strong performance in standard testing does not necessarily translate to success in real-world scenarios. Specifically, in the context of car-following modeling, it is critical to validate a model through driving behavior simulations rather than relying solely on acceleration predictions based on test data. Given the NN model's collision incidents and subsequent simulation failure, it will be excluded



(a) Vehicle acceleration from prediction.

(b) Vehicle acceleration from simulation.

Figure 9: Vehicle acceleration of, a) predicted using testing data, and b) simulated using the trained model as a controller.

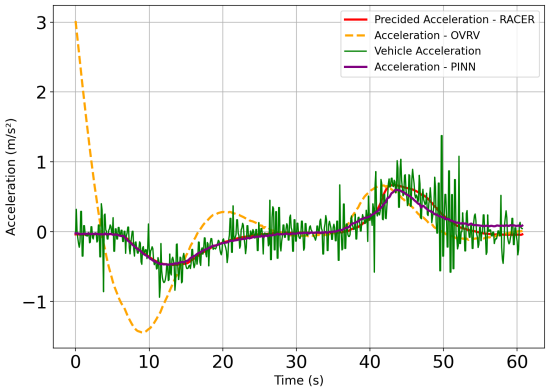


Figure 10: Comparison of vehicle acceleration from different simulation models over time.

from the following discussion. The RMSE for the NN model diverges significantly from the other models, indicating a disparity in predictive fidelity.

For the maximum gap setting, the OVRV model's acceleration predictions have the highest RMSE (0.614 m/s^2), significantly exceeding the lowest recorded RMSE of 0.221 m/s^2 by the RACER model. The RMSE of the PINN model stands at a competitive 0.227 m/s^2, closely trailing RACER and demonstrating consistent performance across various driving conditions, including acceleration and deceleration phases, as shown in Fig. 10. Nonetheless, the speed and spacing predictions by both the PINN and OVRV models do not match the accuracy of the RACER model, with the spacing RMSE being approximately an order of magnitude greater and the speed RMSE at least twice as large. This discrepancy is a testament to the challenges posed by maximum gap setting conditions and suggests that model efficacy during training does not always translate to dependable control capabilities.

As presented from Fig. 10 through 12, the OVRV model exhibits comparatively poor performance, particularly at the onset of the simulation. Notably, the spacing maintained by the OVRV model is less than that of both the RACER model and the originally observed spacing, raising potential safety concerns if used as a vehicle controller. The PINN model performs better than the OVRV model but still shows a divergence in the later stages of the simulation. This divergence contributes to its elevated RMSE in both speed and spacing metrics.

Fig. 13 to Fig. 15 showcase the RDC violations for the ACC vehicle with the maximum setting. Consistent with the earlier results from the minimum setting comparison, the proposed RACER model is the only model that fully complies with the RDCs. In contrast, both the baseline NN and PINN models exhibit violations in one or more of the constraints. Specifically, the NN model fails in this scenario, leading to crashes and irrational predictions. These

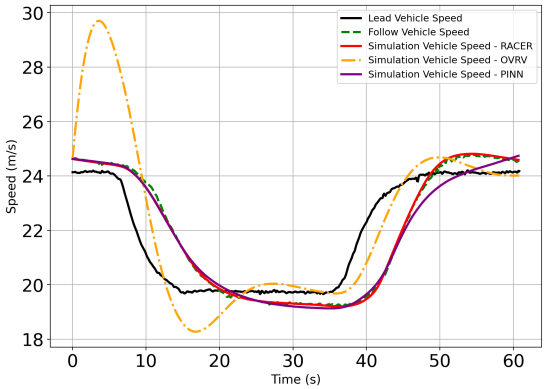


Figure 11: Comparison of vehicle velocity from different simulation models over time.

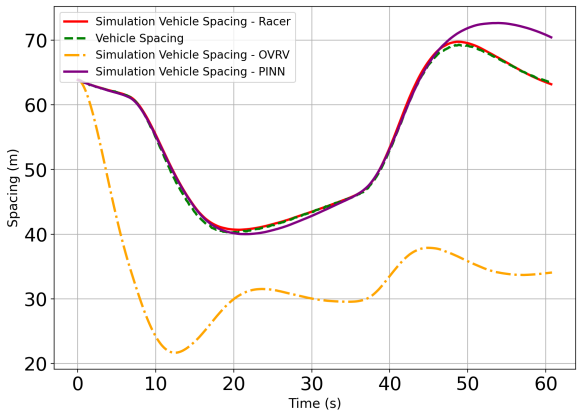


Figure 12: Comparison of vehicle spacing from different simulation models over time.

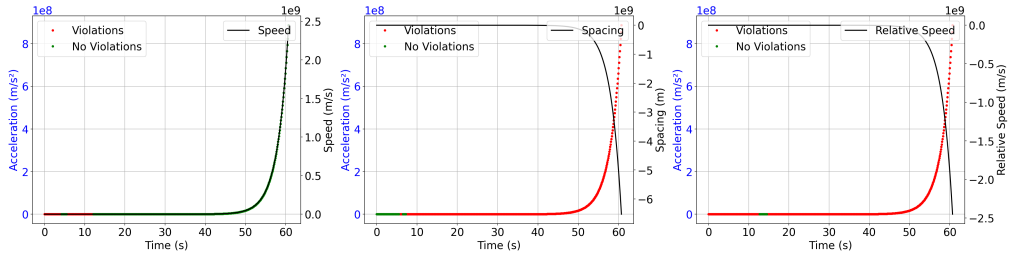
results further emphasize the advantage of incorporating RDCs into the learning process, as demonstrated by the superior performance of the RACER model.

6.4 Model Evaluation: ACC Vehicle with Smoother Acceleration

Table 5: Root mean squared errors for different models used as car-following controllers with smoother acceleration.

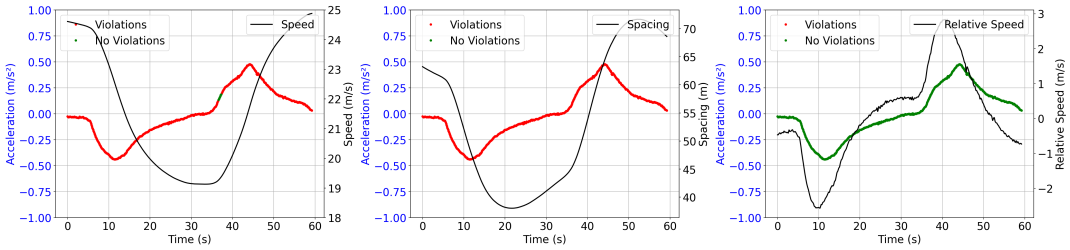
	RACER	OVRV	NN	PINN
Acceleration (m/s^2)	**0.099**	0.111	0.115	0.111
Speed (m/s)	**0.152**	0.173	0.237	0.322
Spacing (m)	**0.298**	1.485	0.559	0.415

In prior experiments, the actual vehicle acceleration was calculated using 0.1-second intervals, which introduced significant noise, as indicated by the high variance in the measurements. This noise obscured the assessment of our proposed RACER model's response to acceleration oscillations, challenging the verification of the model's rationality. To address this, we adopted a larger time interval of 0.5 seconds for a smoother estimation of the vehicle's average acceleration over that that longer time step with the minimum gap setting. Fig. 16 below illustrates the performance of various models. Notably, between 20 and 25 seconds, the RACER model uniquely adapts to the fluctuations in



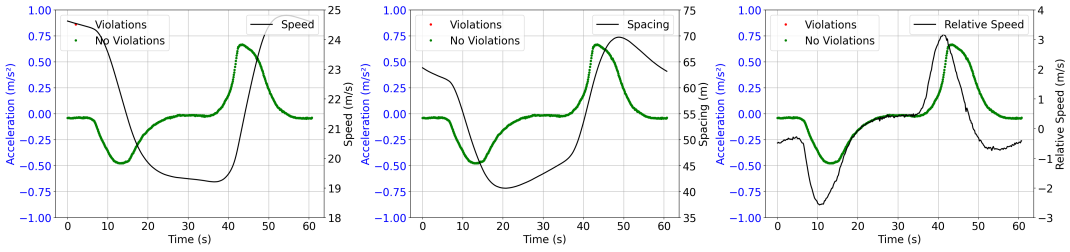
(a) RDCs Violations - Speed (b) RDC Violations - Spacing (c) RDC Violations - Relative Speed

Figure 13: Illustrations of predictions and violations by the LSTM Neural Network model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.



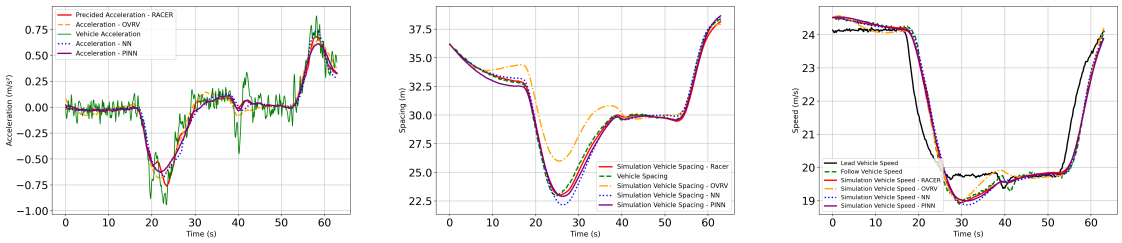
(a) RDCs Violations - Speed (b) RDC Violations - Spacing (c) RDC Violations - Relative Speed

Figure 14: Illustrations of predictions and violations by the physics-informed Neural Network model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.



(a) RDCs Violations - Speed (b) RDCs Violations - Spacing (c) RDCs Violations - Relative Speed

Figure 15: Illustrations of predictions and violations by the RACER model in terms of speed, spacing, and relative speed. The green and red dots denote predictions conforming to and violating the established rules, respectively.



(a) Model Performance - Acceleration (controller) **(b) Model Performance - Spacing (controller)** **(c) Model Performance - Speed (controller)**

Figure 16: Comparison of model performance with respect to acceleration, spacing, and speed.

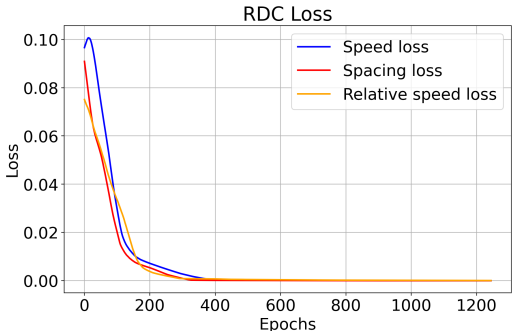


Figure 17: Training progression of RDCs loss, detailing speed, spacing, and relative speed loss components.

acceleration, influenced by the RDCs active during this period. Overall, the RACER model consistently surpasses competing models across all evaluated metrics, including acceleration, spacing, and speed as presented in Table 5.

In addition to the performance metrics discussed, the training process of the RACER model incorporates the minimization of the RDCs loss, which is crucial for maintaining safe and stable car-following behavior. Fig. 17 presents the RDCs loss progression throughout the training epochs. The loss is segmented into three components: speed loss, spacing loss, and relative speed loss. These components are critical as they correspond to the key aspects of rational car-following dynamics that our model aims to optimize.

During training, an evident and rapid decrease in RDC loss is observed, showcasing the model's ability to learn and adapt to the desired behavior efficiently. The convergence of the loss values indicates the model's proficiency in capturing the intrinsic patterns of vehicle dynamics for adaptive cruise control. It is particularly noteworthy that after an initial steep descent, the loss curves flatten out, suggesting that the model reaches a state of minimal error, which aligns with the desired outcome of the RACER model's learning process.

As Table 6 shows, the RACER model uniquely combines the predictive power of deep learning with the hard safety bounds imposed by Rational Driving Constraints. Unlike the OVRV model, RACER captures complex driving patterns; unlike the NN model, it ensures physically meaningful outputs; and unlike the PINN model, it achieves strict RDC compliance rather than relying on a soft physics residual. This combination of high accuracy and provable driving rationality is particularly valuable for ACC vehicles and other safety-critical applications, where even occasional irrational predictions could have hazardous consequences. The present study therefore represents an initial step toward seamlessly incorporating domain-specific knowledge into AI-based car-following models.

7 Discussion & Conclusion

Our experimental findings highlight the significant differences between pure neural network models and their physically constrained counterparts, particularly with respect to handling nonlinear problems and offering predictions that surpass those provided by conventional models like OVRV. These results emphasize the critical importance of integrating physical constraints into neural networks used for acceleration predictions to ensure realistic long-term forecasts. In contrast to existing approaches, our hybrid model bridges the gap between engineering and data-driven methods,

Table 6: Comparison of Car-Following Modeling Approaches.

Characteristic	Physics-Based Models	Neural Networks	PINN	RACER
Foundation	Mathematical equations with physical principles	Data-driven learning from trajectories	Hybrid approach combining NN with physics models	NN with rational driving constraints
Physics Integration	Hard-coded in model structure	None (learned implicitly)	Physics loss term in training	Derivative constraints in loss
RDC Compliance	Directly calculable through PDEs of explicit equations	Not guaranteed	Not directly enforced	Explicitly enforced
RDC Performance	Theoretically verifiable but limited model flexibility	High accuracy but potential violations	Good accuracy with some violations	High accuracy with zero violations
Safety Guarantees	Theoretical through model structure	None	Partial through physics guidance	Complete through RDC enforcement

combining the reliability of physical constraints with the adaptability of machine learning. This hybrid approach enables the model to maintain the integrity of real-world driving principles while leveraging data to optimize its predictions. While recent literature, including studies like [52, 53]), has emphasized the calibration of car-following models based on spacing, our approach introduces a novel perspective by focusing on acceleration as the control output. This decision is grounded in real-world applications, where controllers rely on acceleration or velocity rather than spacing to dictate vehicle behavior. As such, our model's calibration aligns with the control dynamics observed in real-world ACC-equipped vehicles, ensuring a more practical and reliable implementation.

While the PINN framework has proven effective in implicitly capturing some of the outcomes of RDCs, our approach offers several key advancements by explicitly incorporating RDCs into the model's loss function. This explicit integration ensures that the model adheres to these constraints throughout the learning process, resulting in more reliable and interpretable predictions. The explicit incorporation of RDCs, as demonstrated in our RACER model, offers enhanced flexibility by allowing the model to learn complex patterns from real-world driving data while maintaining adherence to fundamental driving constraints. This capability is essential for accurately capturing the nuanced behaviors observed in real-world driving scenarios—behaviors that purely physics-based models may struggle to represent due to their simplified mathematical formulations. The zero RDC violations achieved by RACER result from the hard constraint enforcement mechanism where gradient-based penalties immediately correct any tendency toward irrational behavior during training. In contrast, other models exhibit systematic violation patterns: NN models violate spacing constraints when trying to minimize prediction error in dense traffic scenarios, while PINN models trade off between data fitting and physics compliance, leading to violations when these objectives conflict.

Implementing a loss function that embodies a physical model such as OVRV in a neural network may inadvertently impair performance by exacerbating divergent behaviors, especially when compared to relying on derivative constraints. Yet, with the judicious incorporation of RDCs into our model, we observed superior accuracy and performance, ultimately outshining other existing models. The trajectories generated by our model depict a more rational and safer driving behavior than those derived from alternative models, thereby boosting safety measures. This suggests that our proposed model can offer a novel paradigm for ACC driving behavior.

Incorporating physical constraints into neural network models has value, especially in accurately simulating the physical world. Our proposed controller aims to ensure the rationality of driving behavior. However, we do not assume that human driving behavior is always rational. This is why we focused our testing on ACC-equipped vehicles, which are automated vehicles that we assume should adhere to rational driving principles. The calibration of the RACER model relies on data from controlled experiments (Gunter et al. [11]), where the leading vehicle's trajectory was pre-planned to elicit specific car-following patterns. While this approach enabled precise evaluation of the model under controlled conditions, it does not fully reflect the complexity of naturalistic driving scenarios. In real-world settings, ACC systems must respond to a variety of unpredictable behaviors, such as sudden braking when approaching a queue, lane changes by surrounding vehicles, or gaps created by diverging vehicles. These scenario-dependent factors may affect the ACC's performance in ways not accounted for in the current dataset. Consequently, while the model performs well within the scope of the tested conditions, its generalizability to more diverse real-world scenarios remains to be fully

assessed. Therefore, the RMSE values reported in the Results Section should be regarded as best-case performance under controlled conditions.

Human driving behavior is undoubtedly more complex and drivers occasionally (and often briefly) deviate from the RDCs. However, our proposed model, which effectively combines physical-guided AI with car-following principles, exhibits potential to inform and guide human drivers, thus making driving safer and more rational. The model also holds promise for testing across a wide range of ACC vehicle driving scenarios as well as human driving situations. To enhance the robustness and applicability of the RACER model, future research should prioritize the collection and integration of naturalistic ACC car-following data. Such data would expose the model to a wider spectrum of driving behaviors, enabling a more comprehensive evaluation and refinement of its performance across varied real-world conditions. This includes scenarios involving unpredictable lead vehicle behavior, lane changes, and other complex traffic interactions that are essential for full real-world deployment. Future research could explore this exciting avenue and contribute to the progressive journey toward achieving safer, smarter, and more efficient transportation systems.

Author Contributions

The authors confirm their contribution to the paper as follows: study conception and design: T. Li and R. Stern; data collection: T. Li; analysis and interpretation of results: T. Li, A. Halatsis, and R. Stern; draft manuscript preparation: T. Li, A. Halatsis, and R. Stern. All authors reviewed the results and approved the final version of the manuscript. The authors do not have any conflicts of interest to declare.

Acknowledgment

This work is supported by the University of Minnesota Center for Transportation Studies through the Transportation Scholar's Program. T. Li acknowledges the support of the Dwight David Eisenhower Graduate Fellowship from the Federal Highway Administration.

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

References

[1] H.-S. Tan, R. Rajamani, and W.-B. Zhang. Demonstration of an automated highway platoon system. In *Proceedings of the 1998 American control conference. ACC (IEEE Cat. No. 98CH36207)*, volume 3, pages 1823–1827. IEEE, 1998.

[2] S. Wang, R. Stern, and M. Levin. Optimal control of autonomous vehicles for traffic smoothing. *IEEE Transactions on Intelligent Transportation Systems*, 23(4):3842–3852, 2022.

[3] Shian Wang, Michael W Levin, and Ryan James Caverly. Optimal parking management of connected autonomous vehicles: A control-theoretic approach. *Transportation Research Part C: Emerging Technologies*, 124:102924, 2021.

[4] S. Learn, J. Ma, K. Raboy, F. Zhou, and Y. Guo. Freeway speed harmonisation experiment using connected and automated vehicles. *IET Intelligent Transport Systems*, 12(5):319–326, 2017.

[5] R. E. Stern, S. Cui, M. L. Delle Monache, R. Bhadani, M. Bunting, M. Churchill, N. Hamilton, R. Haulcy, H. Pohlmann, F. Wu, B. Piccoli, B. Seibold, J. Sprinkle, and D. B. Work. Dissipation of stop-and-go waves via control of autonomous vehicles: Field experiments. *Transportation Research Part C: Emerging Technologies*, 89:205 – 221, 2018.

[6] A. Talebpour and H. S. Mahmassani. Influence of connected and autonomous vehicles on traffic flow stability and throughput. *Transportation Research Part C: Emerging Technologies*, 71:143–163, 2016.

[7] M. Shang and R. E. Stern. Impacts of commercially available adaptive cruise control vehicles on highway stability and throughput. *Transportation Research Part C: Emerging Technologies*, 122:102897, 2021.

[8] Alireza Talebpour and Hani S Mahmassani. Influence of autonomous and connected vehicles on stability of traffic flow. Technical report, 2015.

[9] L. C. Davis. Effect of adaptive cruise control systems on traffic flow. *Physical Review E*, 69(6):066110, 2004.

[10] L. C. Davis. The effects of mechanical response on the dynamics and string stability of a platoon of adaptive cruise control vehicles. *Physica A: Statistical Mechanics and its Applications*, 392(17):3798–3805, 2013.

[11] G. Gunter, D. Gloudemans, R. E Stern, S. McQuade, R. Bhadani, M. Bunting, Maria L. Delle M., R. Lysecky, B. Seibold, J. Sprinkle, et al. Are commercially implemented adaptive cruise control systems string stable? *IEEE Transactions on Intelligent Transportation Systems*, 2020.

[12] Mingfeng Shang, Benjamin Rosenblad, and Raphael Stern. A novel asymmetric car following model for driver-assist enabled vehicle dynamics. *IEEE Transactions on Intelligent Transportation Systems*, 23(9):15696–15706, 2022.

[13] V. Milanés and S. E. Shladover. Modeling cooperative and autonomous adaptive cruise control dynamic responses using experimental data. *Transportation Research Part C: Emerging Technologies*, 48:285–300, 2014.

[14] D. C. Gazis, R. Herman, and R. B. Potts. Car-following theory of steady-state traffic flow. *Operations Research*, 7(4):499–505, 1959.

[15] Xiao Wang, Rui Jiang, Li Li, Yilun Lin, Xinhu Zheng, and Fei-Yue Wang. Capturing car-following behaviors by deep learning. *IEEE Transactions on Intelligent Transportation Systems*, 19(3):910–920, 2017.

[16] Sakda Panwai and Hussein Dia. Neural agent car-following models. *IEEE Transactions on Intelligent Transportation Systems*, 8(1):60–70, 2007.

[17] Meixin Zhu, Xuesong Wang, and Yinhai Wang. Human-like autonomous car-following model with deep reinforcement learning. *Transportation research part C: emerging technologies*, 97:348–368, 2018.

[18] Zhaobin Mo, Rongye Shi, and Xuan Di. A physics-informed deep learning paradigm for car-following models. *Transportation research part C: emerging technologies*, 130:103240, 2021.

[19] Htet Naing, Wentong Cai, Hu Nan, Wu Tiantian, and Yu Liang. Dynamic data-driven microscopic traffic simulation using jointly trained physics-guided long short-term memory. *ACM Transactions on Modeling and Computer Simulation*, 32(4):1–27, 2022.

[20] Tianyi Li and Raphael Stern. Car-following-response based vehicle classification via deep learning. *ACM J. Auton. Transport. Syst.*, June 2023.

[21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep learning. *Nature*, 521(7553):436–444, 2016.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[23] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[24] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.

[25] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[26] M. Treiber, A. Hennecke, and D. Helbing. Congested traffic states in empirical observations and microscopic simulations. *Physical Review E*, 62(2):1805, 2000.

[27] Rui Jiang, Qingsong Wu, and Zuojin Zhu. Full velocity difference model for a car-following theory. *Physical Review E*, 64(1):017101, 2001.

[28] Lijing Ma, Shiru Qu, Lijun Song, Zhiteng Zhang, and Jie Ren. A physics-informed generative car-following model for connected autonomous vehicles. *Entropy*, 25(7):1050, 2023.

[29] Kangrui Ruan, Junzhe Zhang, Xuan Di, and Elias Bareinboim. Causal imitation learning via inverse reinforcement learning. In *The Eleventh International Conference on Learning Representations*, 2022.

[30] R. E. Wilson and J. A. Ward. Car-following models: fifty years of linear stability analysis–a mathematical perspective. *Transportation Planning and Technology*, 34(1):3–18, 2011.

[31] M. Brackstone and M. McDonald. Car-following: a historical review. *Transportation Research Part F: Traffic Psychology and Behaviour*, 2(4):181–196, 1999.

[32] Dirk Helbing. Traffic and related self-driven many-particle systems. *Reviews of modern physics*, 73(4):1067, 2001.

[33] M. Treiber and A. Kesting. *Traffic Flow Dynamics*. Springer Berlin Heidelberg, 2013.

[34] D. Gazis, R. Herman, and R. Rothery. Nonlinear follow-the-leader models of traffic flow. *Operations research*, 9(4):545–567, 1961.

[35] Masako Bando, Katsuya Hasebe, Akihiro Nakayama, Akihiro Shibata, and Yuki Sugiyama. Dynamical model of traffic congestion and numerical simulation. *Physical review E*, 51(2):1035, 1995.

[36] Vincenzo Punzo, Maria Teresa Borzacchiello, and Biagio Ciuffo. Influence of aggressiveness on the stochastic distribution of time headway: Psycho-physical modeling and empirical evidence. *Transportation Research Part F: Traffic Psychology and Behaviour*, 14(5):333–347, 2011.

[37] Arne Kesting, Martin Treiber, and Dirk Helbing. Variable speed limits: An overview. *IET Intelligent Transport Systems*, 2(4):261–271, 2008.

[38] Carl-Johan Hoel, Julian Wolff, Bastian Lütjens, Eike Rehder, and Christoph Stiller. Automated driving: The role of forecasts and uncertainty - a control perspective. *Energy Informatics*, 1(1):1–11, 2018.

[39] Xintao Yan, Zhengxia Zou, Shuo Feng, Haojie Zhu, Haowei Sun, and Henry X Liu. Learning naturalistic driving environment with statistical realism. *Nature Communications*, 14(1):2037, 2023.

[40] Shuo Feng, Haowei Sun, Xintao Yan, Haojie Zhu, Zhengxia Zou, Shengyin Shen, and Henry X Liu. Dense reinforcement learning for safety validation of autonomous vehicles. *Nature*, 615(7953):620–627, 2023.

[41] Pratama Mahadika, Aries Subiantoro, and Benyamin Kusumoputro. Neural network predictive control approach design for adaptive cruise control. *Int. J. Technol*, 11:1451, 2020.

[42] Wojciech Samek, Thomas Wiegand, and Klaus-Robert Müller. Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models. *arXiv preprint arXiv:1708.08296*, 2017.

[43] Merry Cherian and S Paul Sathiyan. Neural network based acc for optimized safety and comfort. *International Journal of Computer Applications*, 42(14):1–4, 2012.

[44] Tianyi Li and Raphael Stern. Classification of adaptive cruise control vehicle type based on car following trajectories. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 1547–1552. IEEE, 2021.

[45] Tianyi Li, Mingfeng Shang, Shian Wang, Matthew Filippelli, and Raphael Stern. Detecting stealthy cyberattacks on automated vehicles via generative adversarial networks. In *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, pages 3632–3637. IEEE, 2022.

[46] Tianyi Li and Raphael Stern. Robustness of vehicle identification via trajectory dynamics to noisy measurements and malicious attacks. In *2022 2nd Workshop on Data-Driven and Intelligent Cyber-Physical Systems for Smart Cities Workshop (DI-CPS)*, pages 36–39. IEEE, 2022.

[47] Tianyi Li, Shian Wang, Mingfeng Shang, Seongjin Choi, and Raphael Stern. A customizable neural network based framework for autonomous vehicle control with human-guided learning. In *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, pages 3064–3069. IEEE, 2024.

[48] Jared Willard, Xiaowei Jia, Shaoming Xu, Michael Steinbach, and Vipin Kumar. Integrating physics-based modeling with machine learning: A survey. *arXiv preprint arXiv:2003.04919*, 1(1):1–34, 2020.

[49] Xiuling Huang, Jie Sun, and Jian Sun. A car-following model considering asymmetric driving behavior based on long short-term memory neural networks. *Transportation research part C: emerging technologies*, 95:346–362, 2018.

[50] Abien Fred Agarap. Deep learning using rectified linear units (relu). *arXiv preprint arXiv:1803.08375*, 2018.

[51] G. Gunter, R. Stern, and D. Work. Modeling adaptive cruise control vehicles from experimental data: model comparison. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)*, pages 3049–3054. IEEE, 2019.

[52] Vincenzo Punzo, Zuduo Zheng, and Marcello Montanino. About calibration of car-following dynamics of automated and human-driven vehicles: Methodology, guidelines and codes. *Transportation Research Part C: Emerging Technologies*, 128:103165, 2021.

[53] Yinglong He, Marcello Montanino, Konstantinos Mattas, Vincenzo Punzo, and Biagio Ciuffo. Physics-augmented models to simulate commercial adaptive cruise control (acc) systems. *Transportation Research Part C: Emerging Technologies*, 139:103692, 2022.

[54] Alex Graves, Navdeep Jaitly, and Abdel-rahman Mohamed. Hybrid speech recognition with deep bidirectional lstm. In *2013 IEEE workshop on automatic speech recognition and understanding*, pages 273–278. IEEE, 2013.

[55] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 6645–6649. IEEE, 2013.

[56] Zhiyong Cui, Ruimin Ke, Ziyuan Pu, and Yinhai Wang. Stacked bidirectional and unidirectional lstm recurrent neural network for forecasting network-wide traffic state with missing values. *Transportation Research Part C: Emerging Technologies*, 118:102674, 2020.

[57] Tianyi Li, Guo-Jun Qi, and Raphael Stern. Taxi utilization rate maximization by dynamic demand prediction: A case study in the city of chicago. *Transportation Research Record*, 2676(4):367–379, 2022.

[58] Vincenzo Punzo and Marcello Montanino. Speed or spacing? cumulative variables, and convolution of model errors and time in traffic flow models validation and calibration. *Transportation Research Part B: Methodological*, 91:21–33, 2016.